```
 ____ ____  __  __ ____  _   _ _____ _____
/ ___/ __ \|  \/  |  _ \| | | |_   _|  ___|
| |   | | | | \  / | |_) | | | | | | |_
| |   | | | |\/| |  __/| | | | | |  _|
| |___| |_| | |  | | |   | |_| | | | |___
 \____\___/|_|  |_|_|    \___/  |_| |_____|
```

# PARADIGMS

EC2 • Lambda • Fargate

// "it works on my machine" is not a valid deployment strategy

# $ cat agenda.txt

```
01. EC2 vs Lambda vs Fargate
02. Stateless vs Stateful
03. When to use each approach
04. Cold starts & scaling patterns
05. Why "works on my machine" fails
```

// ~15 min | Questions welcome anytime

```
$ ./deploy.sh --env=local
✓ Build successful
✓ Tests passed
✓ "It works on my machine!"
$ ./deploy.sh --env=prod
✗ ERROR: Everything is on fire
✗ CRITICAL: CEO is calling
✗ FATAL: Update your LinkedIn
```

# $ cat /reasons/it-fails.log

```
[ENV] Different OS, deps, configs
[VAR] That env variable you forgot
[MEM] Your machine: 64GB, Prod: 512MB
[DEP] "Just install Node v47 real quick"
```

// Solution: Reproducible compute environments

# $ ls /aws/compute/

drwxr-xr-x

**EC2/**

// The OG

-rwxr-xr-x

**Lambda**

// The Hipster

drwxr-xr-x

**Fargate/**

// Goldilocks

# $ man ec2

```
NAME

 ec2 - Elastic Compute Cloud


DESCRIPTION

 "Here's a computer. Good luck."
```

# $ ec2 --describe

```
--control Full (and full responsibility)
--patches Your problem
--uptime 24/7 (wallet.drain())
--use-case "I need SSH, don't ask"
```

# $ man lambda

**NAME**

lambda - Serverless Functions

**DESCRIPTION**

"Servers exist. You just pretend they don't."

# $ lambda --describe

```
--billing Per 1ms (seriously)
--scale ∞* (*T&C apply)
--timeout 15min max, then death
--use-case Event-driven, bursty loads
```

# $ man fargate

## NAME

fargate - Serverless Containers

## DESCRIPTION

"docker run, but make it someone else's problem"

# $ fargate --describe

```
--servers None (for you)
--input Your Dockerfile
--timeout None. Run forever.
--use-case Lambda too short, EC2 too much
```

# $ diff ec2 lambda fargate

| | EC2 | Lambda | Fargate |
|---|---|---|---|
| Control | root@ | none | container |
| Scale | manual/ASG | auto-magic | auto |
| Billing | always-on | per-req | per-task |
| Cold Start | ~minutes | ms→seconds | ~seconds |

# $ cat /arch/state.md

```
## STATELESS
// Finding Nemo brain
> No memory
> Easy to scale
> Kill without guilt
```

```
## STATEFUL
// Elephant brain
> Remembers things
> Sticky sessions
> Hard to scale
```

# $ cat examples/stateless.log

```
// STATELESS: REST API

# Request 1 → Server A

GET /api/users/42 → { id: 42, name: "Bob" }

# Request 2 → Server B (different!)

GET /api/users/42 → { id: 42, name: "Bob" }

✓ Token validated each request | ✓ Data from DB | ✓ Any server
```

# $ cat examples/stateful.log

```
// STATEFUL: WebSocket Game Server

# Connection → Server A

WS /game/room/abc

In memory: player, room, inventory

✗ Server dies → session lost | ✗ Can't migrate | ✗ Sticky sessions
```

# $ diff stateless stateful

```
Stateless   Nothing in memory → DB/Redis = truth
Stateful    Data in memory → Memory = truth
Stateless   Crash? No problem, spin new instance
Stateful    Crash? Data gone, user angry
Pro tip: Externalize state to Redis/ElastiCache
```

# $ sudo rm -rf /state/*

```
Stateless apps are like goldfish:
[1] No memory of previous requests
[2] Store state elsewhere (DB/Redis/S3)
[3] Any instance handles any request
[4] kill -9 without drama
```

# $ ./cold-start.sh

```
// The awkward pause before your function
responds


Loading runtime.........
```

# $ time cold-start --all

```
Lambda: 100ms → 10s (runtime dependent)
Fargate: 30s → 2min (image pull)
EC2:    1min → 5min (whole OS boot)
```

// Fix: warm pools, optimize images, provisioned concurrency

# $ top --scaling

```
HORIZONTAL More instances (the way)
VERTICAL Bigger instance (the $ way)
LAMBDA  "What's scaling?" (auto)
```

// Pro tip: Vertical scaling = ur doing it wrong

# $ which compute

```
EC2     → Legacy, GPU, "I need sudo"
Lambda  → APIs, events, cron, <15min
Fargate → Microservices, batch, containers
```

# $ ./decide.sh

```
Need root access? → goto EC2
Short, event-driven? → goto Lambda
Have Dockerfiles? → goto Fargate
Still confused? → goto Lambda
```

# $ cat README.md | head

```
EC2       Max control, max responsibility
Lambda    Pay-per-use, auto-scale, limits
Fargate   Best of both for containers
Stateless The way to scale
Cold      Know them. Plan for them.
```

```
$ echo $DEPLOYMENT_STRATEGY


ERROR: "works on my machine"
≠
"works"


// That's why containers & DevOps exist
```

# $ await questions()

// Press Ctrl+C to exit

© 2026 rishad1234 | github.com/rishad1234/cml-slides